

Example 1:
Input: $n = 5$, $arr = [1, 3, 2, 4, 5]$
Output: $5, 4, 2, 3, 1$

Function Description
Complete the `reverseArray` function in the editor below.
`reverseArray` has the following parameter(s):

`int arr[n]` an array of integers

Return

`int[n]` the array in reverse order

Constraints

$1 \leq n \leq 100$

$0 < arr[i] \leq 100$

Input Format For Custom Testing

The first line contains an integer, n , the number of elements in `arr`.

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer, `arr[i]`.

Sample Case 0

Sample Input For Custom Testing

5
1
3
2
4
5

Sample Output

5
4
2
3
1

Explanation

The input array is `[1, 3, 2, 4, 5]`, so the reverse of the input array is `[5, 4, 2, 3, 1]`.

Sample Case 1

Sample Input For Custom Testing

4
17
10
21
45

Sample Output

45
21
10
17

Explanation

The input array is `[17, 10, 21, 45]`, so the reverse of the input array is `[45, 21, 10, 17]`.

21

10

17

Explanation

The input array is [17, 10, 21, 45], so the reverse of the input array is [45, 21, 10, 17].

Answer: (penalty regime: 0 %)

Reset answer

```

1  /*
2   * Complete the 'reverseArray' function below
3   *
4   * The function is expected to return an integer array.
5   * The function accepts INTEGER_ARRAY arr and integer arr_count as parameters.
6   */
7
8  /*
9   * To return the integer array from the function, you should:
10  *   - Store the size of the array to be returned in the *result_count parameter.
11  *   - Allocate the array statically or dynamically.
12  *
13  * For example,
14  * int* return_integer_array_using_static_allocation(int arr_count) {
15  *     *result_count = 5;
16  *
17  *     static int a[5] = {1, 2, 3, 4, 5};
18  *
19  *     return a;
20  * }
21  *
22  * int* return_integer_array_using_dynamic_allocation(int arr_count) {
23  *     *result_count = 5;
24  *
25  *     int *a = malloc(5 * sizeof(int));
26  *
27  *     for (int i = 0; i < 5; i++) {
28  *         *(a + i) = i + 1;
29  *     }
30  *
31  *     return a;
32  * }
33  */
34
35 #include<stdio.h>
36 #include<stdlib.h>
37 int* reverseArray(int arr_count, int *arr, int *result_count) {
38     int*result = (int*)malloc(arr_count*sizeof(int));
39     if(result ==NULL)
40     {
41         return NULL;
42     }
43     for (int i=0;i<arr_count;i++)
44     {
45         result[i]=arr[arr_count-i-1];
46     }
47     *result_count=arr_count;
48     return result;
49 }
50
51

```

Test

✓ int arr[] = {1, 3, 2, 4, 5};
 int result_count;
 int* result = reverseArray(5, arr, &result_count);
 for (int i = 0; i < result_count; i++)
 printf("%d\n", *(result + i));

Passed all tests! ✓

The first line contains an integer, n , the number of elements in $lengths$.

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer, $lengths[i]$.

The next line contains an integer, $minLength$, the minimum length accepted by the machine.

Sample Case 0

Sample Input For Custom Testing

STDIN	Function
-------	----------

4	→ $lengths[]$ size $n = 4$
3	→ $lengths[] = [3, 5, 4, 3]$
5	
4	
3	
9	→ $minLength = 9$

Sample Output

Possible

Explanation

The uncut rod is $3 + 5 + 4 + 3 = 15$ units long. Cut the rod into lengths of $3 + 5 + 4 = 12$ and 3 . Then cut the 12 unit piece into lengths 3 and $5 + 4 = 9$. The remaining segment is $5 + 4 = 9$ units and that is long enough to make the final cut.

Sample Case 1

Sample Input For Custom Testing

STDIN	Function
-------	----------

3	→ $lengths[]$ size $n = 3$
5	→ $lengths[] = [5, 6, 2]$
6	
2	
12	→ $minLength = 12$

Sample Output

Impossible

Explanation

The uncut rod is $5 + 6 + 2 = 13$ units long. After making either cut, the rod will be too short to make the second cut.

Answer: (penalty regime: 0 %)

Reset answer

Sample Case 1

Sample Input For Custom Testing

STDIN Function

```
3  → lengths[] size n = 3
5  → lengths[] = [5, 6, 2]
6
12 → minLength = 12
```

Sample Output

Impossible

Explanation

The uncut rod is $5 + 6 + 2 = 13$ units long. After making either cut, the rod will be too short to make the second cut.

Answer: (penalty regime: 0 %)

Reset answer

```
1  /*
2   * Complete the 'cutThemAll' function below
3   *
4   * The function is expected to return a string.
5   * The function accepts following parameters:
6   * 1. LONG_INTEGER_ARRAY lengths
7   * 2. LONG_INTEGER minLength
8   */
9
10 /*
11 * To return the string from the function
12 *
13 * For example,
14 * char* return_string_using_static_allocation() {
15 *     static char s[] = "static allocation of string";
16 *
17 *     return s;
18 * }
19 *
20 * char* return_string_using_dynamic_allocation() {
21 *     char* s = malloc(100 * sizeof(char));
22 *
23 *     s = "dynamic allocation of string";
24 *
25 *     return s;
26 * }
27 *
28 */
29 #include<stdio.h>
30 char* cutThemAll(int lengths_count, long
31 long t=0,i=1;
32 for(int i=0;i<lengths_count-1;i++){
33     t+=lengths[i];
34 }
35 do{
36     if(t-lengths[lengths_count-1]<minLength)
37         return "Impossible";
38 }
39 i++;
40 }while(i<lengths_count-1);
41 return "Possible";
42 }
43
44 }
45
```

Test	Expected Output
✓ long lengths[] = {3, 5, 4, 3}; printf("%s", cutThemAll(4, lengths, 9))	Possible
✓ long lengths[] = {5, 6, 2}; printf("%s", cutThemAll(3, lengths, 12))	Impossible

Passed all tests! ✓